ITE 031 - Data Structures and Algorithms

Comprehensive Project Documentation: Printer Queue Simulation

Java (Console-Based Application)

Members

Resty S. Aldave

John Michael L. Cabilao

SWU-BSIT2-B1

SY 2026 – 2027

TABLE OF CONTENTS

1 · · · · · · · · · · · · · **PROJECT PROPONENTS & ROLES**

2 · · · · · · · · · · · · · **OVERVIEW**

3 · · · · · · · · · · · · · **DATA STRUCTURES & ALGORITHMS IMPLEMENTATION**

- 3.1. Queue
- 3.2. List (ArrayList)
- 3.3. Temporary Collection Snapshot Pattern
- 3.4. Immutable List (List.of) and Fixed Array (String[ ])
- 3.5. Linear Character Membership Search
- 3.6. Number-Based Menu Selection
- 3.7. Selection Sort Algorithm

7 · · · · · · · · · · · · · **FUNCTIONALITIES**

- 7.1.File Metadata Validation & Instantiation (FileDetails Constructor)
- 7.2. Number-Based Menu Selection (Printer)
- 7.3. Print Job Submission (Printer.addPrintJob)
- 7.4. Canceling a Print Job (Printer.selectJobToRemove & Printer.removePrintJob)
- 7.5. Print Simulation & Non-Blocking Interruption
- 7.6. Queue Monitoring Views (Printer.viewFullQueue & Printer.quickQueueView)
- 7.7. Printed Archive Management & Sorted Reporting
- 7.8. Application Bootstrap & Master Control Loop (PrintingQueueSimulation.main)

16 · · · · · · · · · · · · · **APPLICATION OF EXCEPTION HANDLING**

- 16.1. Keeping the Terminal Working via finally Blocks (Printer)
- 16.2. Thread Interruption Management & Status Preservation (Printer & FileDetails)

18 · · · · · · · · · · · · · **PROJECT LEARNING OUTCOME**

PROJECT PROPONENTS & ROLES

Proponent	Role	Responsibilities
Resty S. Aldave	System Developer / Data Structure Lead	Program design, Java class structure, Queue data structure implementation, menu logic, and input validation.
John Michael L. Cabilao	Algorithm, Testing & Documentation Lead	Algorithm design, pseudocode, flowcharts, test case formulation, error tracking, and project documentation.

OVERVIEW

PrintingQueue Simulation is a console based Java application that models how a physical printer manages incoming document print jobs sequentially. Using a First-In, First-Out (FIFO) queue, the program ensures documents are processed in the exact order they arrive, while offering an intuitive number-based menu for straightforward console navigation.

To keep the system reliable, the application includes strict input validation that trims extra whitespace, blocks forbidden symbols, enforces realistic page limits (1–500 pages), and lets users choose from preset file extensions. Users can monitor active tasks at any time using either a full queue breakdown or a quick summary count, as well as cancel specific waiting jobs before printing starts.

During execution, the program simulates an animated boot-up sequence and live, page-by-page printing with timed delays, allowing users to stop an active job mid-process by pressing Enter. Once finished, completed documents move automatically to a history archive, where an in-place selection sort organizes them alphanumerically (0–9, A–Z) for clear post-print review. The entire system is built with defensive error handling to prevent crashes, keep screen output clean, and ensure a smooth shutdown.

DATA STRUCTURES AND ALGORITHMS

1. Queue (Queue using LinkedList)

The program uses a queue to handle pending print files. Just like a real printer line, documents need to be printed in the exact order they were added first in, first out. Using a linked list makes this very fast because the computer can add files to the end or take them off the front without having to slide the rest of the items around.

- First-Come, First-Served: Makes sure the oldest files are always printed first.
- Fast Adding and Removing: Adding a new file to the back or taking a finished one from the front happens instantly.
- Peeking Ahead: Lets the program check the file's name and page count while keeping it safely in line until it is done printing.

2. List (ArrayList)

A dynamic list is used to save all the files that have already finished printing. Unlike the active print line, this finished history doesn't need to follow a strict line order; it just needs an easy way to store finished files, show them on screen, and sort them alphabetically.

- Easy to Read and Swap: The program can grab or swap any item by its position number, which makes sorting the list simple.
- Quick Saving: Adding a completed file to the end of the history takes almost no time.
- Grows Automatically: Expands on its own as more files finish, so there is no fixed limit to worry about.

3. Temporary Collection Snapshot Pattern

When you want to remove a file from the line, the program makes a quick copy of the queue into a list. A normal queue only lets you look at the very front item, so you can't pick item #3 or #4 directly. Copying it into a list first gives every file a number so you can choose which one to delete.

- Numbered Choices: Lets the screen show a numbered list of files so you can easily pick one.
- Safe Deletion: Finds the exact file matching the number you typed and removes it from the queue.
- Protects the Line: Keeps the original queue working as a normal line without breaking its order.

4. Immutable List (List.of) and Fixed Array (String[ ])

Things that never change like allowed file extensions (txt, png, docx) and the main menu titles are stored in fixed lists and arrays. Because these options stay the same the whole time the program runs, using fixed lists keeps the code safe and uses very little memory.

- Cannot Be Changed: Prevents the program from accidentally changing or deleting valid file types.
- Lightweight: Uses very little memory because it only holds what it needs.
- Set Size: The count of items is always known, making it easy to display them correctly.

5. Linear Character Membership Search

To make sure file names don't use bad symbols like %, $, or *, the program checks the name one letter at a time against a blocked list. It is a straightforward check that doesn't need any complex tools.

- Simple and Fast: Scans each character directly without slowing down the program.
- Instant Stop: Stops checking and alerts you the second it finds a bad character.
- No Extra Setup: Uses simple built-in checks, keeping the code clean and lightweight.

6. Number-Based Menu Selection

The program uses simple number choices to run different tasks. Every action on the screen has a number next to it (like 1 to 7). The user just types a number and presses Enter to pick what they want to do. The program checks the number and immediately opens that feature.

- Quick and Direct: Users type a number (like 1 to add a file or 3 to start printing) to jump straight to what they need.
- Input Checking: If the user types a number not on the list, the program catches it and asks them to try again.
- Easy to Use: Uses standard typing and the Enter key, running smoothly on any computer without extra setup.

7. Selection Sort Algorithm

Finished print jobs are organized from 0 to Z using an in-place selection sort. The program looks through the list of completed files, finds the next name in alphabetical and numeric order (ignoring uppercase vs. lowercase), and swaps it into its correct spot.

- Sorts in Place: Reorders items directly inside the existing list without needing extra storage space.
- Fewer Swaps: Only swaps items when necessary, keeping memory changes to a minimum.
- Simple and Self-Contained: Handles alphabetical order and filenames starting with numbers neatly without needing extra tools or libraries.

FUNCTIONALITIES

1. File Metadata Validation & Instantiation (FileDetails Constructor)

This feature handles setting up a new file before it gets added to the print line. It asks the user for the file name, trims off extra spaces, and makes sure the name is not left blank or filled with forbidden symbols. Once the name is approved, the user picks a file type, and the program makes sure the page count is a whole number between 1 and 500 pages.

- Cleans User Input: Removes extra spaces with trim() and blocks blank or empty names.
- Blocks Bad Symbols: Checks the name against forbidden characters (%$.,^+=*!) and asks you to try again if one is found.
- Sets Type and Page Limits: Lets you pick the file extension and makes sure the page count is a valid number from 1 to 500.

```
    Scanner input = new Scanner(System.in);

    //Added new class to separate the getting of file details...
    String fileName;
    String fileType;
    int filePages = -1;

    private static final String INVALID_CHARS = "%$.,^+=*!";

    private static boolean hasInvalidChar(String name) {
        for (int i = 0; i < name.length(); i++) {
            if (INVALID_CHARS.indexOf(name.charAt(i)) >= 0) {
                return true;
            }
        }
        return false;
    }

    public FileDetails(){
        String name;
        while (true) {
            System.out.print("\nEnter the file name: ");
            name = input.nextLine().trim();

            if (name.isEmpty()) {
                System.out.println("\n*Invalid input! File name cannot be empty. Returning to the menu...*");
                return;
            }

            if (hasInvalidChar(name)) {
                System.out.println("\n*Invalid input! File name must not contain any of these characters: % $ . , ^ + = * !*");
                Printer.pause(1000);
                Printer.clearScreenStatic();
                continue;
            }

            break;
        }

        this.fileName = name;

```

```
            this.fileType = Printer.selectFileTypeStatic(fileName);

            // Keep asking for the page count until a valid whole number is typed
            while (true) {
                Printer.clearScreenStatic();
                System.out.println("\nFile name: " + getFullFileName());
                System.out.print("\nHow many pages does it have: ");

                String line = input.nextLine().trim();
                try {
                    this.filePages = Integer.parseInt(line);
                    break;
                } catch (NumberFormatException e) {
                    System.out.println("\n*Invalid input! Please enter a whole number for the page count.*");
                    Printer.pause(1000);
                }
            }
        }

        // Mergg the details
        public String getFullFileName(){
            return fileName + "." + fileType;
        }

        public int getFilePages(){
            return filePages;
        }
}
```

2. Number-Based Menu Selection (Printer)

This feature handles menu displays and user choices using standard console input. It prints each option with a corresponding number, prompts the user to type their choice, and reads the input directly. This approach keeps navigation straightforward, eliminates platform-specific terminal commands, and works reliably on any operating system without additional libraries.

- Clear Numbered Options: Displays all available choices with numbers so the user knows what to type.
- Direct Input Reading: Reads choices using standard console input without requiring complex terminal modes.
- Cross-Platform Compatibility: Runs seamlessly across Windows, macOS, and Linux without native system calls or terminal resets.

```
150
151     //no more up and down
152     public static int selectMenuOption(String[] options) {
153         Runnable draw = () -> {
154             clearScreenStatic();
155             System.out.println("-----------Printing Simulation-----------");
156             System.out.println("\n*Type the number of your choice, then press Enter.*\n");
157             for (int i = 0; i < options.length; i++) {
158                 System.out.println("     " + (i + 1) + ". " + options[i]);
159             }
160         };
161
162         // same here it draw again if action is incorrect
163         draw.run();
164         return readNumber(options.length, draw);
165     }
```

3. Print Job Submission (Printer.addPrintJob)

This module instantiates a new print document, validates its state, and enqueues it onto the active spooler. It inspects the fields returned by the FileDetails constructor to confirm that file creation was not aborted midway, verifies page count limits (enforcing a ceiling of 500 pages), and appends the valid job object to the tail of the LinkedList queue.

- Defensive Integrity Checks: Verifies that filenames are non-null and non-empty, and confirms pages were assigned.
- Volume Constraints: Enforces business logic bounding valid page counts between 1 and 500.
- Tail Enqueueing: Adds valid jobs directly to printingQueue in time.

```
166
167
168     public void addPrintJob(){
169         FileDetails newFile = new FileDetails();
170
171         // connected sa checking input text... ug checking name is empti
172         if (newFile.fileName == null || newFile.fileName.isEmpty() || newFile.getFilePages() == -1) {
173             return;
174         }
175
176         //added a limit para dili usahon ang >500 pages
177         if (newFile.getFilePages() <= 0 || newFile.getFilePages() > 500) {
178             System.out.println("\n*Invalid page count! A print job cannot have 0 or more than 500 pages. Please try again.*");
179             return;
180         }
181
182         printingQueue.add(newFile);
183         System.out.print("\n*The file '" + newFile.getFullFileName());
184         System.out.println("' with " + newFile.getFilePages() + " pages has been successfully added!*");
185     }
186
187
```

4. Canceling a Print Job (Printer.selectJobToRemove & Printer.removePrintJob)

This feature lets users pick and cancel a specific print job before it starts printing. Because you cannot pick items by position in a regular queue, the program makes a temporary list copy so every waiting file gets a number. Users can then easily choose which file to remove or pick an option to go back to the menu without changing anything.

- Numbered List of Jobs: Copies waiting files into a temporary list so each one has a number you can see and choose.
- Option to Cancel: Adds a "Back to Menu" option at the end so you can back out without deleting any files.
- Direct Removal: Finds the exact file you selected and takes it out of the print line safely.

```
187
188     //same here no more use of up and down
189     public int selectJobToRemove(){
190          java.util.List<FileDetails> jobs = new java.util.ArrayList<>(printingQueue);
191          int totalOptions = jobs.size() + 1;
192
193          Runnable draw = () -> {
194               clearScreenStatic();
195               System.out.println("-----------Remove Print Job-----------");
196               System.out.println("\n*Type the number of the file to remove, then press Enter.*\n");
197               for (int i = 0; i < jobs.size(); i++) {
198                    System.out.println("     " + (i + 1) + ". " + jobs.get(i).getFullFileName() + " | Pages: " + jobs.get(i).getFilePages());
199               }
200               System.out.println("\n     " + totalOptions + ". Back to Menu");
201          };
202
203          // draw if incorrect
204          draw.run();
205          int n = readNumber(totalOptions, draw);
206          if (n == totalOptions) {
207               return -1;
208          }
209          return n - 1;
210     }
211
212
213     public void removePrintJob(){
214          if(printingQueue.isEmpty()){
215               System.out.println("\n*Queue is empty. No job to remove.*");
216          } else {
217               int index = selectJobToRemove();
218               clearScreen();
219
220               if(index == -1){
221                    System.out.println("\n*Remove cancelled. Returning to the menu...*");
222                    return;
223               }
224
225               FileDetails removed = new java.util.ArrayList<>(printingQueue).get(index);
226               printingQueue.remove(removed);
227               System.out.println("\n*Removed job: " + removed.getFullFileName() + "*");
228          }
229     }
```

5. Print Simulation & Non-Blocking Interruption

This feature mimics a real printer by printing documents page by page with short time pauses. It shows a live page counter on the screen that updates in place rather than printing new lines over and over. If you need to stop printing, pressing Enter cancels the job right away without freezing the screen. Once a file finishes, it moves to the finished history list, and the program asks if you want to print the next file in line.

- Live Page Counter: Updates the current page count on the same line so the screen stays clean and easy to read.
- Stop by Pressing Enter: Checks for an Enter press between pages to stop the print job immediately if needed.
- Keeps Files Safe While Printing: Holds the document at the front of the line while printing and only removes it once every page is done.
- Asks Before Continuing: Asks if you are ready to print the next file whenever multiple documents are waiting in line.

```
244     // for asking if to continue with the next queue to print
245     private boolean askStartNext(FileDetails next){
246         while(true){
247             System.out.print("\n*The next print \"" + next.getFullFileName() + "\" will be started (y/n)?* ");
248             String answer = input.nextLine().trim();
249 
250             if(answer.equalsIgnoreCase("y")){
251                 return true;
252             }
253             if(answer.equalsIgnoreCase("n")){
254                 return false;
255             }
256             System.out.println("\n*Invalid input! Please enter y or n.*");
257             pause(1000);
258             clearScreen();
259             System.out.println("\n*Print job completed successfully!*");
260         }
261     }
262 
```

```
263
264     public void startPrintJob(){
265         if(printingQueue.isEmpty()){
266             System.out.println("\n*No print jobs in the queue.*");
267             return;
268         }
269
270         stopPrinting = false;
271         System.out.println("\n*Printing Started! (Press Enter to stop printing)*");
272
273         try {
274             while(!printingQueue.isEmpty() && !stopPrinting){
275                 FileDetails currentJob = printingQueue.peek();
276                 int total = currentJob.getFilePages();
277
278                 System.out.println("\nPrinting: " + currentJob.getFullFileName() + " with a total " + total + " pages.");
279
280                 for(int i = 1; i <= total; i++){
281                     if(enterPressed()){
282                         stopPrinting = true;
283                         break;
284                     }
285                     System.out.print("\rPrinted page " + i + "/" + total + "          ");
286                     System.out.flush();
287                     TimeUnit.MILLISECONDS.sleep(500);
288                 }
289
290                 System.out.println();
291                 FileDetails finishedJob = printingQueue.poll();
292
293                 if(stopPrinting){
294                     System.out.println("\n*Print was abruptly stopped by user.*");
295                     break;
296                 }
297
298                 printedFiles.add(finishedJob);
299                 System.out.println("\n*Print job completed successfully!*");
300
301                 if(!printingQueue.isEmpty()){
302                     if(!askStartNext(printingQueue.peek())){
303                         System.out.println("\n*Returning to the menu...*");
304                         break;
305                     }
306                     System.out.println("\n*Moving to the next print job...*");
307                     TimeUnit.MILLISECONDS.sleep(1000);
308                     clearScreen();
309                 }
310             }
311         } catch (InterruptedException e) {
312             System.out.println("\n*Printing was interrupted!*");
313             return;
314         }
315
316         if(printingQueue.isEmpty() && !stopPrinting){
317             System.out.println("\n*All queued print jobs have been processed.*");
318         }
319     }
320
```

6. Queue Monitoring Views (Printer.viewFullQueue & Printer.quickQueueView)

This feature lets you see what files are currently waiting in line to be printed. You can choose to view the complete list which shows every file's name, type, and total page count in the exact order they were added or quickly check just how many files are waiting in line without cluttering the screen.

- Safe Reading: Looks through all the waiting files in order to show their details without removing or altering anything in the line.
- Two Ways to View: Gives you a detailed breakdown of every file with viewFullQueue() or a fast summary count with quickQueueView().

```
322     public void viewFullQueue(){
323         if(printingQueue.isEmpty()){
324             System.out.println("\n*The printing queue is empty.*");
325         } else {
326             System.out.println("\n--- Full Printing Queue ---\n");
327             int count = 1;
328             for(FileDetails job : printingQueue){
329                 System.out.println(count + ". " + job.getFullFileName() + " | Pages: " + job.getFilePages());
330                 count++;
331             }
332         }
333     }
334
335     public void quickQueueView(){
336         System.out.println("\n*Queueing " + printingQueue.size() + " of files. Ready to print.*");
337     }
338
```

7. Printed Archive Management & Sorted Reporting

This module manages the history of completed jobs. Before display, it is used to sort all completed documents alphabetically and numerically by full filename using an in-place selection sort. It then prints the ordered list with final page counts.

- Pre-Render Alphabetization:Before display to guarantee A–Z case-insensitive ordering, and numeric from 0-9.
- Empty State Handling: Detects empty histories and prints an informative notice instead of an empty table.

```
340      //sorting 0 - z
341      public void sortPrintedFiles(){
342           for(int i = 0; i < printedFiles.size() - 1; i++){
343                int minIndex = i;
344                for(int j = i + 1; j < printedFiles.size(); j++){
345                     if(printedFiles.get(j).getFullFileName().compareToIgnoreCase(printedFiles.get(minIndex).getFullFileName()) < 0){
346                          minIndex = j;
347                     }
348                }
349                FileDetails temp = printedFiles.get(i);
350                printedFiles.set(i, printedFiles.get(minIndex));
351                printedFiles.set(minIndex, temp);
352           }
353      }
354
355
356      public void viewPrintedFiles(){
357           if(printedFiles.isEmpty()){
358                System.out.println("\n*No files have been printed yet.*");
359           } else {
360                sortPrintedFiles();
361                System.out.println("\n--- Previously Printed Files (A-Z) ---\n");
362                int count = 1;
363                for(FileDetails job : printedFiles){
364                     System.out.println(count + ". " + job.getFullFileName() + " | Pages: " + job.getFilePages());
365                     count++;
366                }
367           }
368      }
369 }
370
```

8. Application Bootstrap & Master Control Loop (PrintingQueueSimulation.main)

This is the starting point of the whole application. When you launch the program, it plays a short loading animation with dots and pauses to look like a real printer turning on. After startup, it keeps the main menu running on repeat, reading your number choices (1 to 7), opening the feature you picked, and giving you time to read the output before closing down cleanly when you choose to exit.

- Startup Simulation: Shows a short loading message with timed pauses to feel like a machine warming up.
- Menu Routing: Takes your number choice (1 through 7) and immediately opens the right feature using a simple switch statement.
- Clean Shutdown: Stops the menu loop smoothly when you pick the Exit option, printing a polite goodbye message before ending the program.

```java
public class PrintingQueueSimulation {

     public static void clearScreen(){
          System.out.print("\033[2J\033[H");
          System.out.flush();
     }

     public static void main (String[] args) {

      //    Printer Booting Up Animation Like (!!!)
          clearScreen();
          System.out.print("\n\n\nPrinter Starting Please Wait");

          int t = 1;
          for (int i = 0 ; i < 3 ; i++){
               System.out.print("!");

               try {
               TimeUnit.SECONDS.sleep(t);
               t++;
               } catch(InterruptedException e){
                    System.out.println("\n*Failed to start!*");
                    System.out.println("\n*Please try again!*");
               }
          }

          Printer job = new Printer();
          job.clearScreen();

          boolean start = true;

          // The looping of the queue printing
          while(start){
               String[] menuOptions = {
                    "Add Print Job",
                    "Remove Print Job",
                    "Start Print Job",
                    "View Printed Files",
                    "View Full Queue",
                    "Quick Queue View",
                    "Exit"
               };

               int choice = Printer.selectMenuOption(menuOptions);
               job.clearScreen();
```

```java
               switch(choice){
                    case 1:
                         job.addPrintJob();
                         break;
                    case 2:
                         job.removePrintJob();
                         break;
                    case 3:
                         job.startPrintJob();
                         break;
                    case 4:
                         job.viewPrintedFiles();
                         break;
                    case 5:
                         job.viewFullQueue();
                         break;
                    case 6:
                         job.quickQueueView();
                         break;
                    case 7:
                         start = false;
                         System.out.println("\n*Thank you for using the Printing Simulation made by Aldave & Cabilao. :/*\n");
                         break;
                    default:
                         System.out.println("\n*Invalid choice! Please choose between 1-7.*");
               }

               if(start){
                    Printer.pause(3000);
               }

          }
     }
}
```

APPLICATION OF EXCEPTION HANDLING

1. Keeping the Terminal Working via finally Blocks (Printer)

The program pauses for a short moment between printing each page so it feels like a real machine at work. While the program is resting, the computer can sometimes interrupt it. The program uses a try...catch block to handle this interruption safely so the app never freezes or crashes

- Safe Stops: If the printing process gets cut off, the program catches the interruption and stops right away instead of breaking down.
- Helpful Alert: Displays a clear message ("*Printing was interrupted!*") so you know the job was stopped.
- Protects Document Order: Returns safely to the rest of the application without losing or messing up the files waiting in line.

```
264     public void startPrintJob(){
265         if(printingQueue.isEmpty()){
266             System.out.println("\n*No print jobs in the queue.*");
267             return;
268         }
269 
270         stopPrinting = false;
271         System.out.println("\n*Printing Started! (Press Enter to stop printing)*");
272 
273         try {
274             while(!printingQueue.isEmpty() && !stopPrinting){
275                 FileDetails currentJob = printingQueue.peek();
276                 int total = currentJob.getFilePages();
277 
278                 System.out.println("\nPrinting: " + currentJob.getFullFileName() + " with a total " + total + " pages.");
279 
280                 for(int i = 1; i <= total; i++){
281                     if(enterPressed()){
282                         stopPrinting = true;
283                         break;
284                     }
285                     System.out.print("\rPrinted page " + i + "/" + total + "          ");
286                     System.out.flush();
287                     TimeUnit.MILLISECONDS.sleep(500);
288                 }
289 
290                 System.out.println();
291                 FileDetails finishedJob = printingQueue.poll();
292 
293                 if(stopPrinting){
294                     System.out.println("\n*Print was abruptly stopped by user.*");
295                     break;
296                 }
297 
298                 printedFiles.add(finishedJob);
299                 System.out.println("\n*Print job completed successfully!*");
300 
301                 if(!printingQueue.isEmpty()){
302                     if(!askStartNext(printingQueue.peek())){
303                         System.out.println("\n*Returning to the menu...*");
304                         break;
305                     }
306                     System.out.println("\n*Moving to the next print job...*");
307                     TimeUnit.MILLISECONDS.sleep(1000);
308                     clearScreen();
309                 }
310             }
311         } catch (InterruptedException e) {
312             System.out.println("\n*Printing was interrupted!*");
313             return;
314         }
315 
316         if(printingQueue.isEmpty() && !stopPrinting){
317             System.out.println("\n*All queued print jobs have been processed.*");
318         }
```

2. Thread Interruption Management & Status Preservation (Printer & FileDetails)

The program uses a dedicated pause helper method to briefly wait during tasks, like showing a message or pacing the simulation. If the computer interrupts the program while it is resting, Java triggers an InterruptedException. Instead of letting this crash the application, the method catches the interruption and handles it cleanly.

- Re-flags the Stop Signal: When a pause is interrupted, Java automatically clears the stop signal. Calling Thread.currentThread().interrupt() turns that signal back on so the rest of the application still knows an interruption happened.
- Crash Prevention: Enclosing Thread.sleep() inside a try...catch block ensures unexpected interruptions never crash the application.
- Reusable and Tidy: Wrapping the pause logic inside a single pause(int ms) function keeps the rest of the codebase clean and easy to read.

```
96      // time pause
97      public static void pause(int ms) {
98          try {
99              Thread.sleep(ms);
100         } catch (InterruptedException e) {
101             Thread.currentThread().interrupt();
102         }
103     }
```

PROJECT LEARNING OUTCOMES

Working on this project really showed us how data structures are actually used in everyday software, not just in textbook examples. Before this, we mostly thought of queues and lists as theoretical concepts, but building a print simulation made it click. We had to think carefully about why a queue makes the most sense for handling incoming documents in a first-come, first-served order, while a list works better when you want to look through or organize files that have already finished printing. It made us realize that picking the right tool from the start saves a lot of headaches later on.

Beyond just storing data, we also learned a lot about how to make a program feel smooth and reliable for the person using it. Dealing with unexpected user inputs, sorting file names neatly by both letters and numbers, and letting someone cancel a print job mid-way were all tricky challenges. It taught us that good programming isn’t just about making the code run when everything goes right, it's about preparing for when things go wrong and making sure the system recovers gracefully instead of freezing or breaking down.

Lastly, this experience taught us the value of collaboration, clean code design, and paying attention to small details that improve the user experience. Breaking the project down into separate classes forced us to think about how different parts of a program talk to each other without creating bugs or clutter. Working through every error together showed us that building software takes a lot of patience, testing, and teamwork. Overall, it gave us real confidence in turning abstract programming lessons into a working, reliable project we can actually be proud of.